Programming

William Schultz

June 24, 2026

1 Concurrency

When trying to program in a concurrent setting with multiple threads, *mutexes* and *condition variables* are often seen as the two core, fundamental concurrency primitives needed to safely manage and coordinate shared state between threads.

In C++, for example, we have `std::mutex` and `std::condition_variable` classes that provide a way to manage and coordinate shared state between threads. A mutex is simply a lock that can be acquired and released by a thread, and can only be held atomically by one thread at a time. A condition variable is always associated with some mutex, and in general there can be multiple condition variables (each corresponding to different waiting predicates) associated with the same mutex. This provides a fundamental way to ensure (1) safe concurrent access to shared state and (2) an efficient way for threads to correctly wait on some condition to become true of the concurrent object before proceeding.

For example, if you want to implement a *bounded concurrent queue* from scratch in a low level language like C++, you start with a basic queue object and protect its accesses by a mutex. This is fine for safe concurrent access, but what happens when a thread wants to push onto a queue that is already full, or pop from an empty queue? You can do this naively by *spin waiting* i.e. simply drop the mutex temporarily (to give others a chance to make progress on the queue), and then re-acquire it and check for the enabling condition again. This is inefficient, though, since it leads to constant wasted CPU cycles for all threads waiting on this condition.

We want a more efficient way to handle this by allowing threads to "sleep" when a condition is not yet held, and be woken up when the condition check becomes true. In essence, a condition variable is really just a queue of waiting threads associated with (1) some mutex and (2) a common predicate/condition they are waiting on. When the condition variable is "signaled", it goes to either one (`notify_one`) or all (`notify_all`) of the threads on this queue and wakes them up, so they can re-acquire the mutex and check their condition before proceeding.

1.1 Hierarchy of Mutexes and Condition Variables

In some ways, we can view a mutex itself as a kind of specialized condition variable, where the condition is not split out explicitly into a separate object, but it implicitly exists as the "mutex is locked" condition. And there is, similarly, a natural set/queue of waiters that exist associated with any mutex which are currently blocked trying to acquire the mutex.

A key point here, though, is that when one of these waiters is woken up, similar to the condition variable mechanics, they can't acquire a mutex to re-check this underlying condition, as that would be circular (we're implementing the mutex itself!). Instead, they just rely on some lower level, fundamental atomic primitive like compare-and-swap (CAS), which essentially implements that same idea but in a core hardware primitive i.e. update the state to locked only of the condition that mutex is not currently locked holds. The idea is the same as what you do with condition variables in general, though i.e. atomically re-check some condition holds and if so update the core piece of state you care about.

So, implementing something like a *bounded concurrent queue* requires you to atomically check full or empty status of queue before updating, and get signalled on appropriate changes, and *reader-writer* lock similarly requires you to maintain some pieces of state around the number of readers or writers currently holding the lock, so you can check this atomically before modifying its state. A basic *mutex* itself can similarly be viewed as a single piece of "locked" boolean state which you must atomically check and update when trying to acquire the lock for yourself. Condition variables then just extend this a bit by making the notion of a waiting queue/set explicit, and with ability to slice up threads based on the specific condition they are waiting on, for efficiency purposes.

```
atomically {
    check condition P
```

```
        if P holds, update state accordingly
        exit atomic block and go to sleep.
    }
```

In a basic mutex, we can simply have a *locked* variable state that we check for being locked or unlocked. For a reader-write lock, we can maintain the number of readers and writers holding the lock, and/or the number of writers waiting. Readers trying to acquire the lock have a certain condition to check (e.g can acquire lock if there are no other writers holding it), and writers trying to acquire the lock have a slightly different condition to check (can acquire lock only if no other readers or writers holding it). All of these basically share the same pattern, though, where you need to atomically check some condition, and then modify state atomically based on that condition check. This is then coupled with appropriate signalling/waking up of a sleeping when the condition they are waiting on may have changed status.

1.2 Reader-Writer Locks

In cases of data structures that require concurrent read/write access patterns (hash maps, databases, etc.), using a single global mutex is often not the most efficient approach. Allowing clients to acquire either a *read lock* (shared) or a *write lock* (exclusive) is more efficient, as many readers can concurrently be holding the lock while reading. Only a single writer may be holding the lock at any time, which blocks out readers from acquiring. From an implementation standpoint, we can build this internally by simply using a standard mutex that protects additional internal state to track the number of readers and/or writers. Basically, we can track the number of readers currently holding the lock, and whether a writer is holding the lock. If we want to decide on a particular fairness policy, we can also choose a *writer preferred* policy, where we track an additional piece of "writer waiting" state and so a reader waiting on the lock is not allowed to acquire until there are no writers waiting.

1.3 Hash Map

Implementing a concurrent hash map can similarly take advantage of read/write access patternsn for locking. More generally, for scalable concurrent access, we can utilize some kind of striping/bucketing/sharding of keys that are each protected by their own locks. Lock-free hash tables built on CAS ops are also possible.

References